



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 17

Dr. Asad Mahmood

Feb. 24, 2026

Lecture Outline

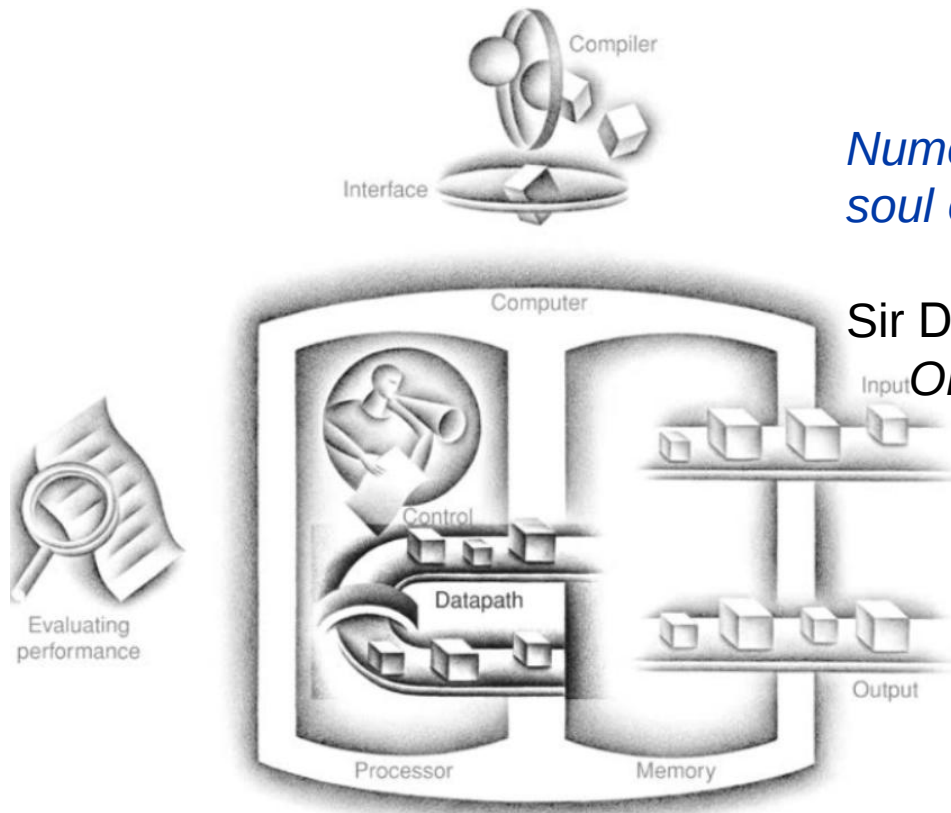
- Announcements ()
- Outline of Previous Lecture:
 - **Chapter 3 – Arithmetic for Computers**
 - 3.3 Multiplication
- Outline to Today's Lecture:
 - **Chapter 3 – Arithmetic for Computers**
 - 3.4 Division

Chapter 3

Arithmetic for Computers

Chapter Theme

The Five Classic Components of a Computer



Numerical precision is the very soul of science.

Sir D'arcy Wentworth Thompson,
On Growth and Form, 1917

Introduction

Numerical precision is the very soul of science.

Sir D'arcy Wentworth Thompson,
On Growth and Form, 1917

Chapter Outline

Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism
- 3.7 Real Stuff: Streaming SIMD Extensions in x86
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply
- 3.9 Fallacies and Pitfalls
- 3.10 Concluding Remarks
- 3.11 Historical Perspective and Further Reading

Introduction

- Computer operates on integers using their **binary** representation
- **Arithmetic operations** are **very important for a processor**
- **Common arithmetic operations on integers**
 - Addition and subtraction
 - **Multiplication** and **division**
 - Dealing with **overflow**
- Dealing with **real numbers**
- **Implications** of operations implementations on ISA

Chapter Outline

Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism
- 3.7 Real Stuff: Streaming SIMD Extensions in x86
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply
- 3.9 Fallacies and Pitfalls
- 3.10 Concluding Remarks
- 3.11 Historical Perspective and Further Reading

Integer Addition

- As studied in logic design
- Example: $7 + 6$
- Overflow
 - When adding +ve and -ve operands
 - When adding two +ve or two -ive operands
 - How to detect?

Integer Subtraction

- Add negation of second operand
- Example: $7 - 6$
- Overflow
 - Subtracting two +ve or two -ve operands
 - Subtracting +ve from -ve operand
 - Subtracting -ve from +ve operand

Arithmetic Operations

- With RISC-V 32-bit registers, we will need an extra bit to detect overflows
- Overflow in unsigned nos.
- Some languages like C may not give correct answer when overflow happens -> Solution?
- Implementations of these arithmetic operations greatly impact processor performance

Arithmetic for Multimedia

- Graphics and media processing operates on vectors of 8-bit and 16-bit data
 - Use 64-bit adder, with partitioned carry chain
 - Operate on 8×8-bit, 4×16-bit, or 2×32-bit vectors
 - SIMD (single-instruction, multiple-data)
- Saturating operations
 - On overflow, result is largest representable value
 - c.f. 2s-complement modulo arithmetic
 - E.g., clipping in audio, saturation in video

Chapter Outline

Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism
- 3.7 Real Stuff: Streaming SIMD Extensions in x86
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply
- 3.9 Fallacies and Pitfalls
- 3.10 Concluding Remarks
- 3.11 Historical Perspective and Further Reading

Multiplication

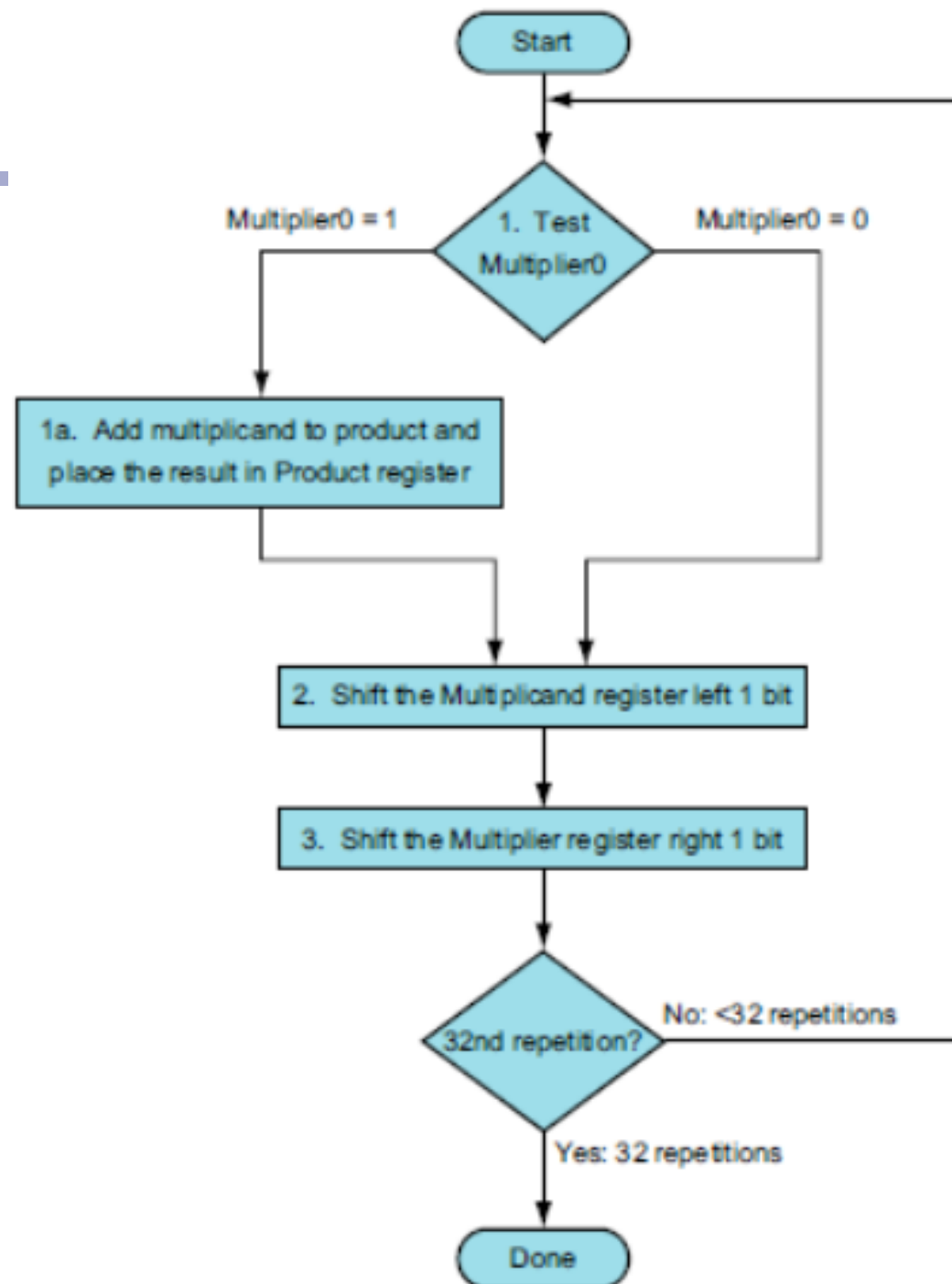
“Multiplication is vexation, Division is as bad; The rule of three doth puzzle me, And practice drives me mad.

Anonymous, Elizabethan
manuscript, 1570

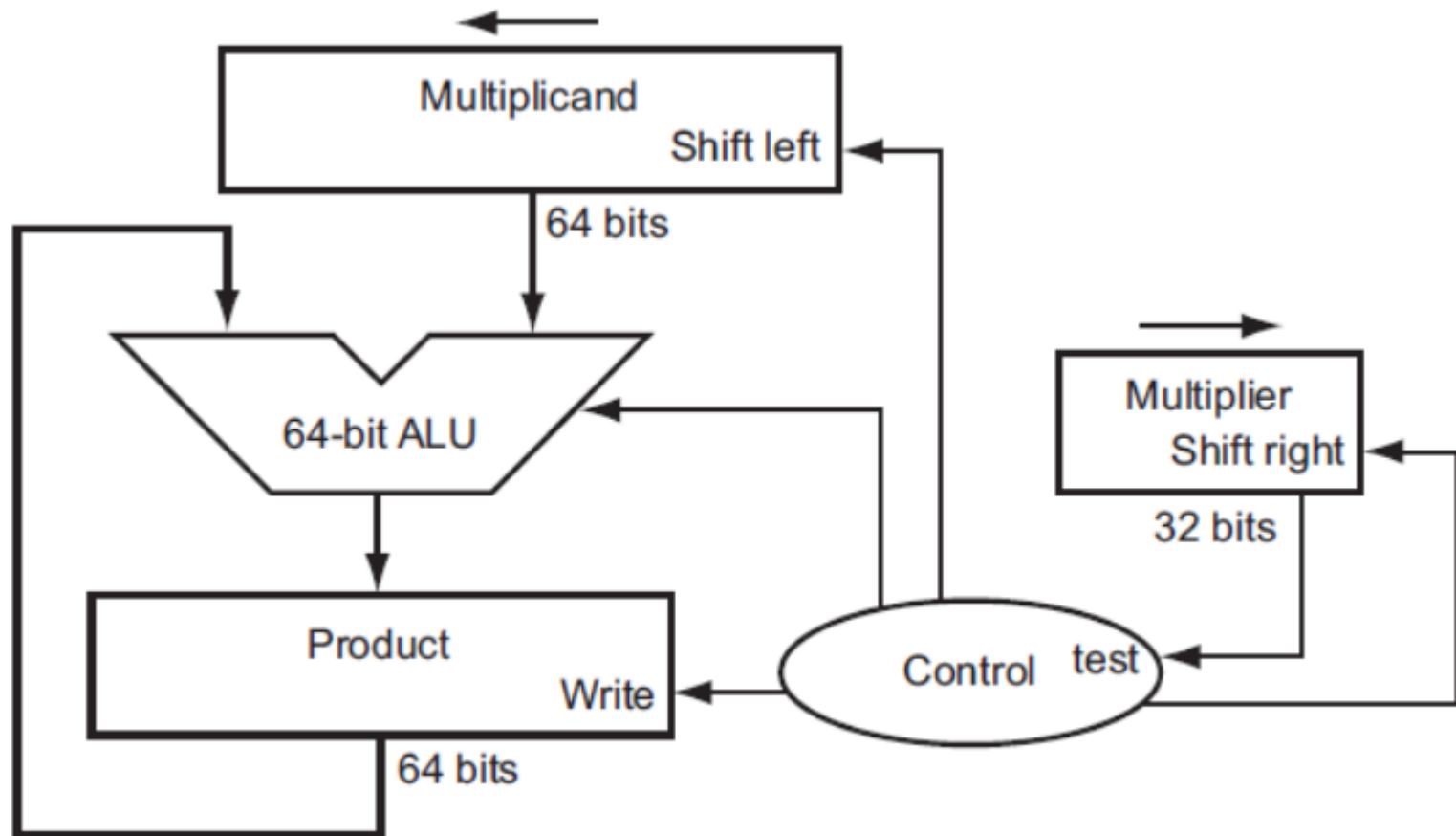
Multiplication

- **How** do we multiply binary numbers?
- **Length of product** is the sum of operand lengths
 - **Implications for RISC-V?**
- We will go through **evolution of binary multipliers** instead of only presenting the most optimized/recent version

Multiplication Algorithm



Multiplication

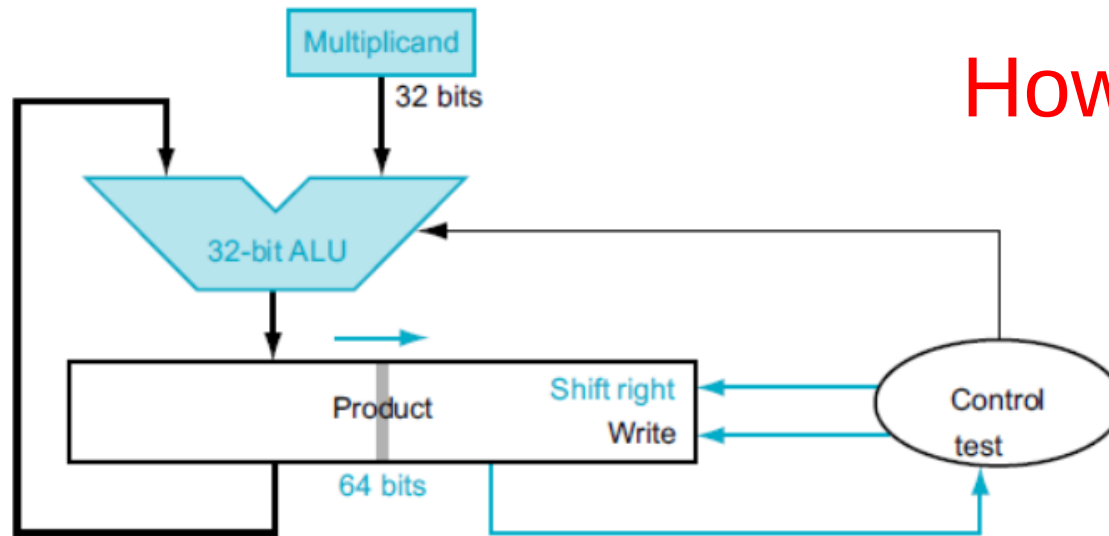


Multiplication Algorithm Example

Iteration	Step	Multiplier	Multiplicand	Product
0	Initial values	001 <u>1</u>	0000 0010	0000 0000
1	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	0011	0000 0010	0000 0010
	2: Shift left Multiplicand	0011	0000 0100	0000 0010
	3: Shift right Multiplier	000 <u>1</u>	0000 0100	0000 0010
2	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	0001	0000 0100	0000 0110
	2: Shift left Multiplicand	0001	0000 1000	0000 0110
	3: Shift right Multiplier	000 <u>0</u>	0000 1000	0000 0110
3	1: $0 \Rightarrow$ No operation	0000	0000 1000	0000 0110
	2: Shift left Multiplicand	0000	0001 0000	0000 0110
	3: Shift right Multiplier	000 <u>0</u>	0001 0000	0000 0110
4	1: $0 \Rightarrow$ No operation	0000	0001 0000	0000 0110
	2: Shift left Multiplicand	0000	0010 0000	0000 0110
	3: Shift right Multiplier	000 <u>0</u>	0010 0000	0000 0110

Optimized Multiplier

- Perform steps in parallel



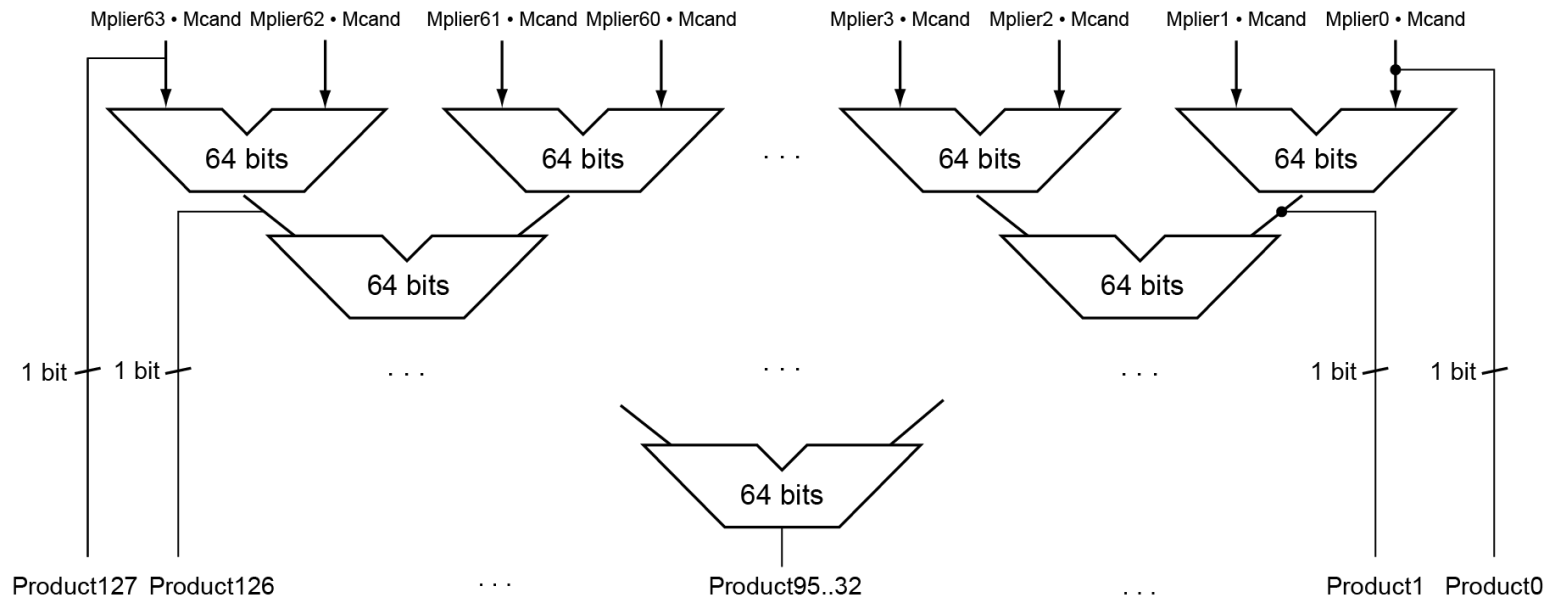
- One cycle per partial-product addition
 - That's ok, if frequency of multiplications is low

Signed Multiplication

- So far, we have only looked at multiplication of positive numbers
- How to multiply signed numbers?

Faster Multiplier

- Uses multiple adders
 - Cost/performance tradeoff



- Can be pipelined
 - Several multiplication performed in parallel

RISC-V Multiplication

- Four multiply instructions:
 - `mul`: multiply
 - Gives the lower 32 bits of the product
 - `mulh`: multiply high
 - Gives the upper 32 bits of the product, assuming the operands are signed
 - `mulhu`: multiply high unsigned
 - Gives the upper 32 bits of the product, assuming the operands are unsigned
 - `mulhsu`: multiply high signed/unsigned
 - Gives the upper 32 bits of the product, assuming one operand is signed and the other unsigned

Chapter Outline

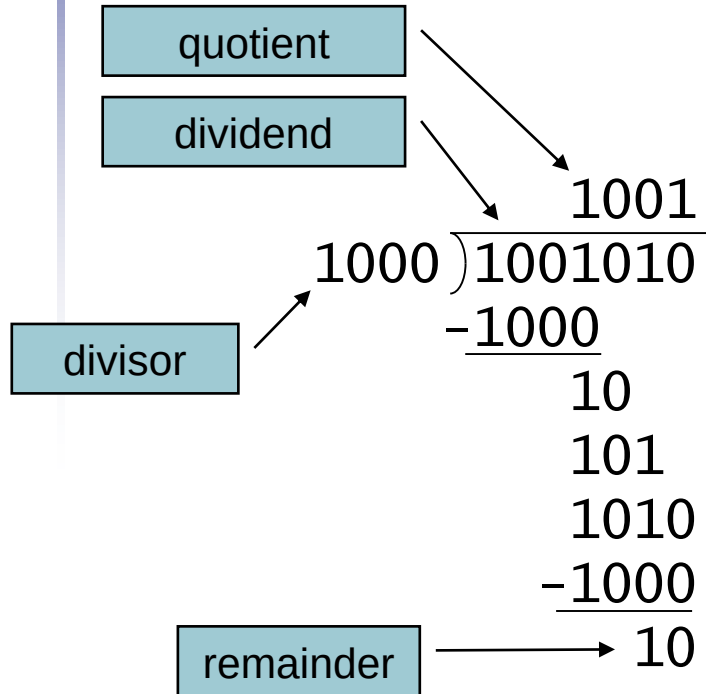
Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism
- 3.7 Real Stuff: Streaming SIMD Extensions in x86
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply
- 3.9 Fallacies and Pitfalls
- 3.10 Concluding Remarks
- 3.11 Historical Perspective and Further Reading

Division

*Divide et impera. Latin for
“Divide and rule,”
ancient political maxim cited by
Machiavelli, 1532*

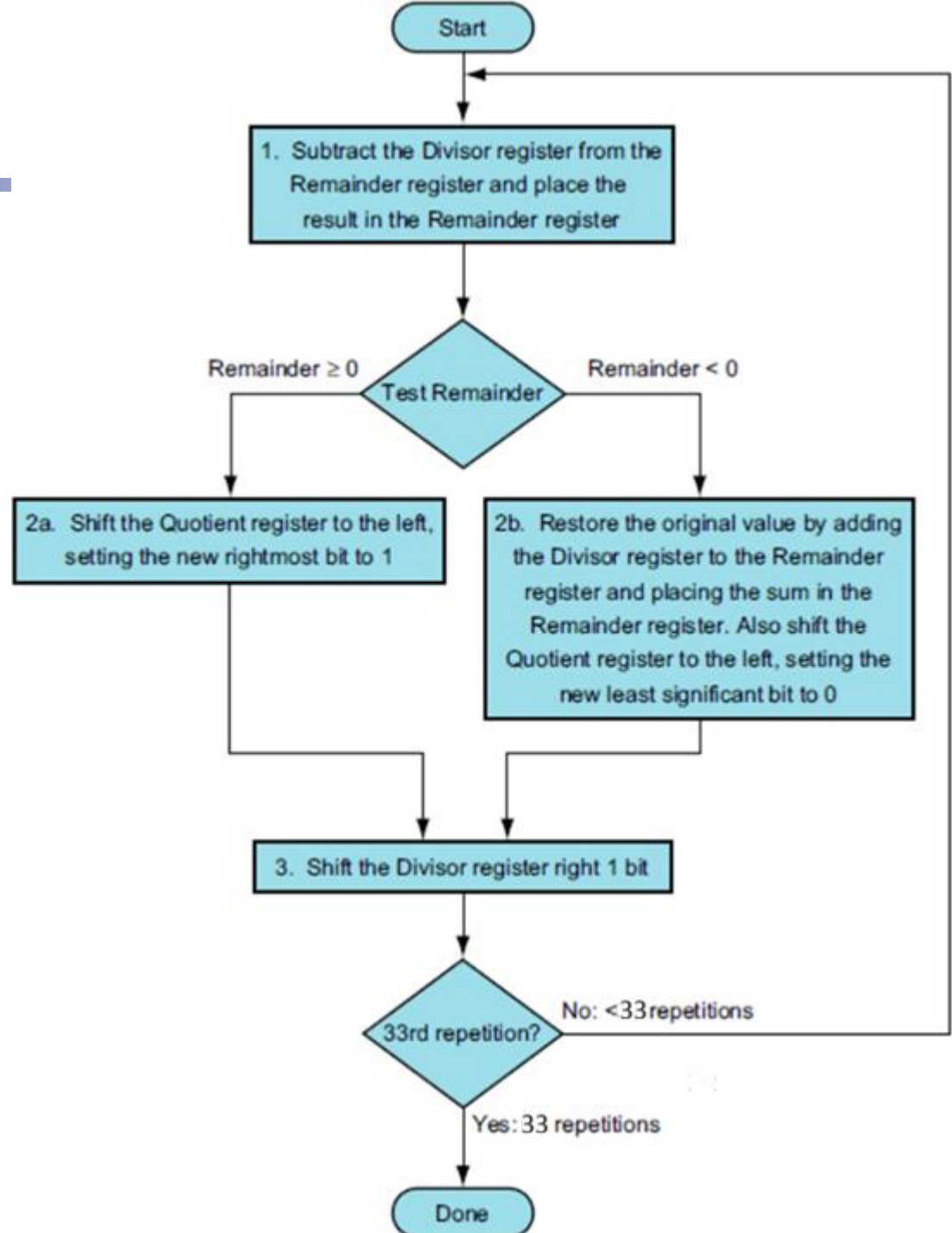
Division



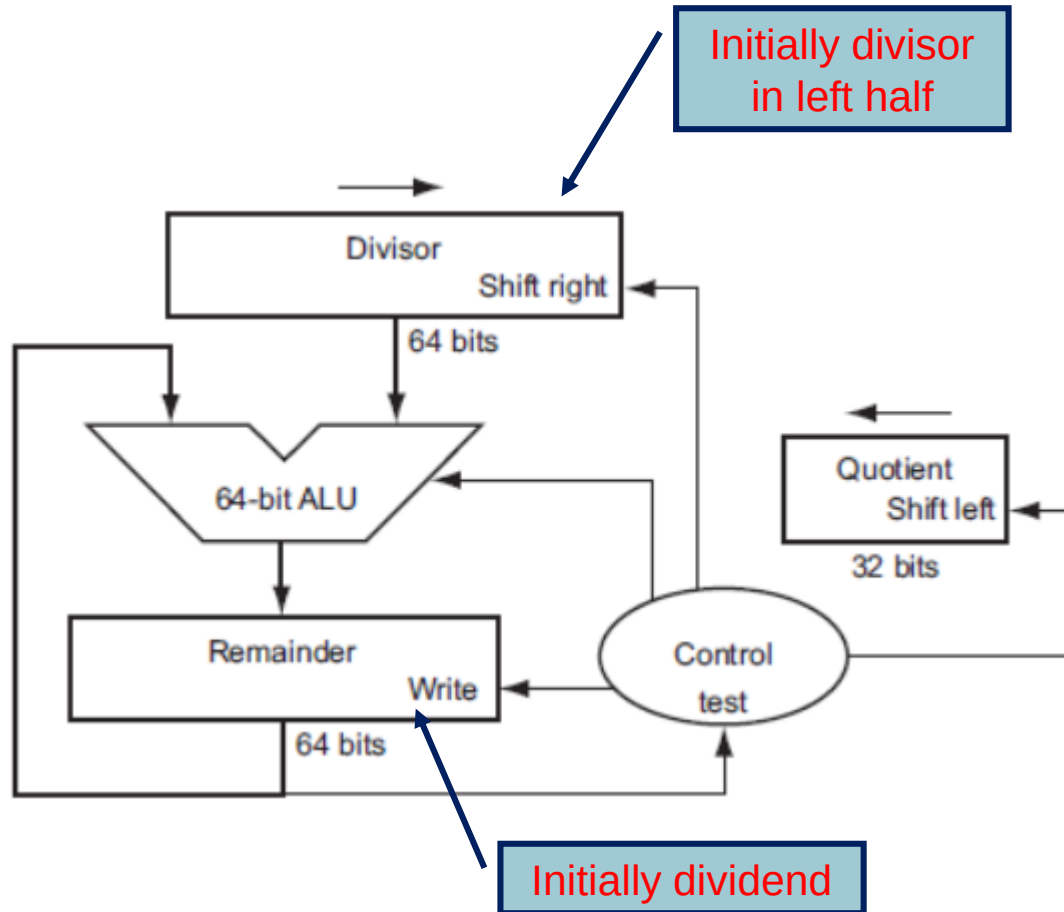
n-bit operands yield *n*-bit quotient and remainder

- More strange than multiplication
- Long division approach
 - If divisor $\leq$ dividend bits
 - 1 bit in quotient, subtract
 - Otherwise
 - 0 bit in quotient, bring down next dividend bit
- Restoring division
 - Do the subtract, and if remainder goes < 0 , add divisor back
- Signed division

Division Algorithm



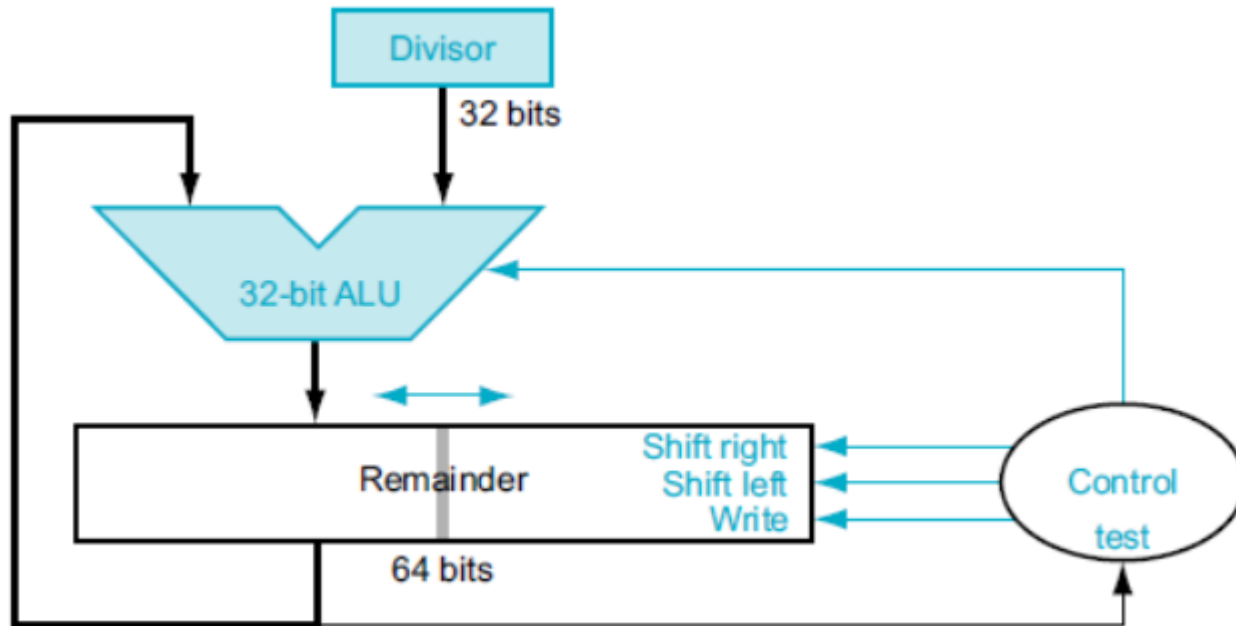
Division Hardware



Division Algorithm Example

Iteration	Step	Quotient	Divisor	Remainder
0	Initial values	0000	0010 0000	0000 0111
1	1: Rem = Rem - Div	0000	0010 0000	①110 0111
	2b: Rem < 0 $\Rightarrow$ +Div, SLL Q, Q0 = 0	0000	0010 0000	0000 0111
	3: Shift Div right	0000	0001 0000	0000 0111
2	1: Rem = Rem - Div	0000	0001 0000	①111 0111
	2b: Rem < 0 $\Rightarrow$ +Div, SLL Q, Q0 = 0	0000	0001 0000	0000 0111
	3: Shift Div right	0000	0000 1000	0000 0111
3	1: Rem = Rem - Div	0000	0000 1000	①111 1111
	2b: Rem < 0 $\Rightarrow$ +Div, SLL Q, Q0 = 0	0000	0000 1000	0000 0111
	3: Shift Div right	0000	0000 0100	0000 0111
4	1: Rem = Rem - Div	0000	0000 0100	①000 0011
	2a: Rem $\geq$ 0 $\Rightarrow$ SLL Q, Q0 = 1	0001	0000 0100	0000 0011
	3: Shift Div right	0001	0000 0010	0000 0011
5	1: Rem = Rem - Div	0001	0000 0010	①000 0001
	2a: Rem $\geq$ 0 $\Rightarrow$ SLL Q, Q0 = 1	0011	0000 0010	0000 0001
	3: Shift Div right	0011	0000 0001	0000 0001

Optimized Divider



- One cycle per partial-remainder subtraction
- Looks a lot like a multiplier!
 - Same hardware can be used for both